Fundamentals of PYTHON DATA STRUCTURES

2ND EDITION

Kenneth A. Lambert

National Economics University
Faculty of Mathematical Economics

EP03.TITH1121 - DATA STRUCTURES AND ALGORITHMS: Queue

- Instructor: Duc Minh Vu (MFE)
- Email: minhvd [at] neu.edu.vn

Learning Objectives

1. Describe the features of a queue and the operations on it
2. Choose a queue implementation based on its performance characteristics
3. Recognize applications where it is appropriate to use a queue
4. Explain the difference between a queue and a priority queue
5. Recognize applications where it is appropriate to use a priority queue

Overview of Queues

- Queues are linear collections in which insertions are restricted to one end, called the **rear**, and removals to the other end, called the **front**.
  - Insert an element to a collection – this new element becomes rear.
  - Remove the oldest element in the collection (the front), the next oldest one becomes front
- Adhere to a **first-in first-out (FIFO) protocol**.
- The operations for putting items on and removing items from a queue are called **push/add/enqueue** and **pop/dequeue.**

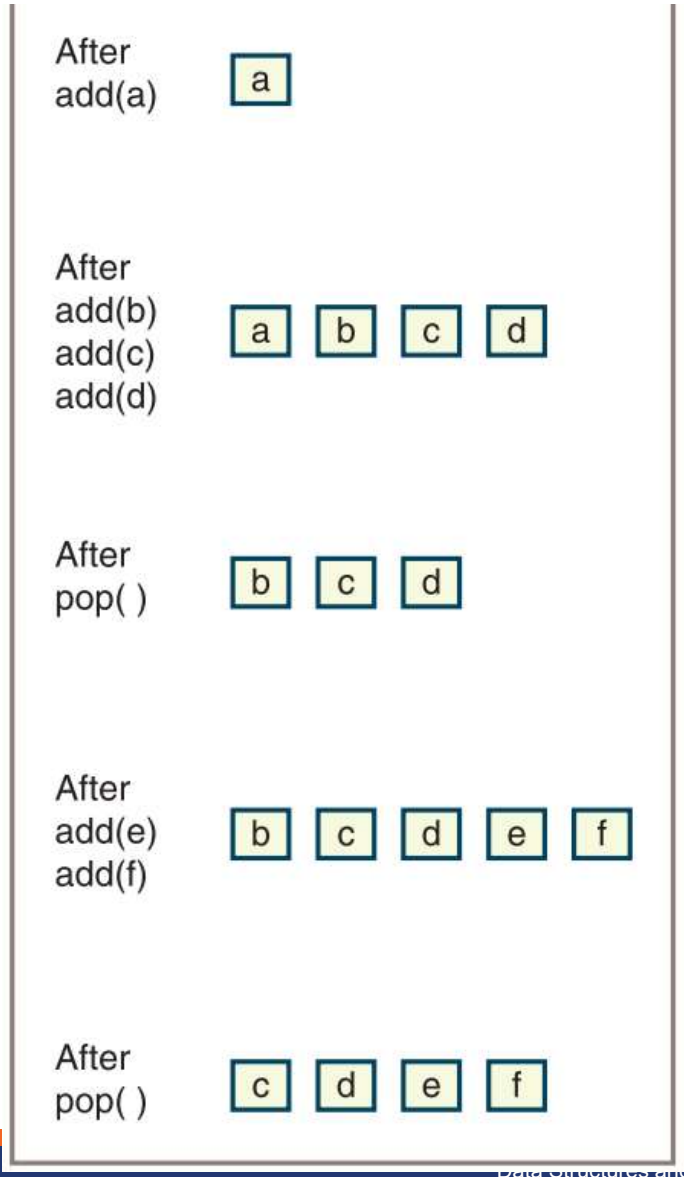

Figure 8-1 The states in the lifetime of a queue

- Initially, the queue is empty. Then an item called **a** is added. Next, three more items called **b**, **c**, and **d** are added, after which an item is popped, and so forth

Some applications

- Translating infix expressions to postfix form and evaluating postfix expressions.
- CPU access—Processes are queued for access to a shared CPU.
- Disk access—Processes are queued for access to a shared secondary storage device.
- Printer access—Print jobs are queued for access to a shared laser printer.

The Queue Interface and Its Use

- The Queue Interface
  - push/add, pop, peek and other basic methods
    - ✓peek = examine the top element but not pop
- Instantiating a Queue
  - q1 = ArrayQueue() # Create empty array queue
  - q2 = LinkedQueue([3, 6, 0]) # Create linked queue with given items

Queue Method	What It Does
`q.isEmpty()`	Returns **`True`** if **`q`** is empty or **`False`** otherwise.
`__len__(q)`	Same as **`len(q)`**. Returns the number of items in **`q`**.
`__str__(q)`	Same as **`str(q)`**. Returns the string representation of **`q`**.
`q.__iter__()`	Same as **`iter(q)`**, or **`for item in q:`**. Visits each item in **`q`**, from front to rear.
`q.__contains__(item)`	Same as **`item in q`**. Returns **`True`** if item is in **`q`** or **`False`** otherwise.
`q1__add__(q2)`	Same as **`q1 + q2`**. Returns a new queue containing the items in **`q1`** followed by the items in **`q2`**.
`q.__eq__(anyObject)`	Same as **`q == anyObject`**. Returns **`True`** if **`q`** equals **`anyObject`** or **`False`** otherwise. Two queues are equal if the items at corresponding positions are equal.
`q.clear()`	Makes **`q`** become empty.
`q.peek()`	Returns the item at the front of **`q`**. *Precondition*: **`q`** must not be empty; raises a **`KeyError`** if the queue is empty.
`q.add(item)`	Adds **`item`** to the rear of **`q`**.
`q.pop()`	Removes and returns the item at the front of **`q`**. *Precondition*: **`q`** must not be empty; raises a **`KeyError`** if the queue is empty.

Table 8-1 The Methods in the Queue Interface

Operation	State of the Queue After the Operation	Value Returned	Comment
`q = <Queue Type>()`			Initially, the queue is empty.
`q.add(a)`	**a**		The queue contains the single item **a**.
`q.add(b)`	**a b**		**a** is at the front of the queue and **b** is at the rear.
`q.add(c)`	**a b c**		**c** is added at the rear.

`q.isEmpty()`	`a b c`	`False`	The queue is not empty.
`len(q)`	`a b c`	`3`	The queue contains three items.
`q.peek()`	`a b c`	`a`	Return the front item on the queue without removing it.
`q.pop()`	`b c`	`a`	Remove the front item from the queue and return it. **b** is now the front item.
`q.pop()`	`c`	`b`	Remove and return **b**.
`q.pop()`		`c`	Remove and return **c**.
`q.isEmpty()`		`True`	The queue is empty.
`q.peek()`		`exception`	Peeking at an empty queue throws an exception.
`q.pop()`		`Exception`	Trying to pop an empty queue throws an exception.
`q.add(d)`	`d`		**d** is the front item.

Table 8-2 The Effects of Queue Operations

Exercises

1. Using the format of Table 8-2, complete a table that involves the following sequence of queue operations.

Operation	State of Queue After Operation	Value Returned
`Create q`		
`q.add(a)`		

Operation	State of Queue After Operation	Value Returned
q.add(b)		
q.add(c)		
q.pop()		
q.pop()		
q.peek()		
q.add(x)		
q.pop()		
q.pop()		
q.pop()		

A Linked Implementation of Queues

- The operation push adds a node at the head of the sequence, whereas add adds a node at the tai

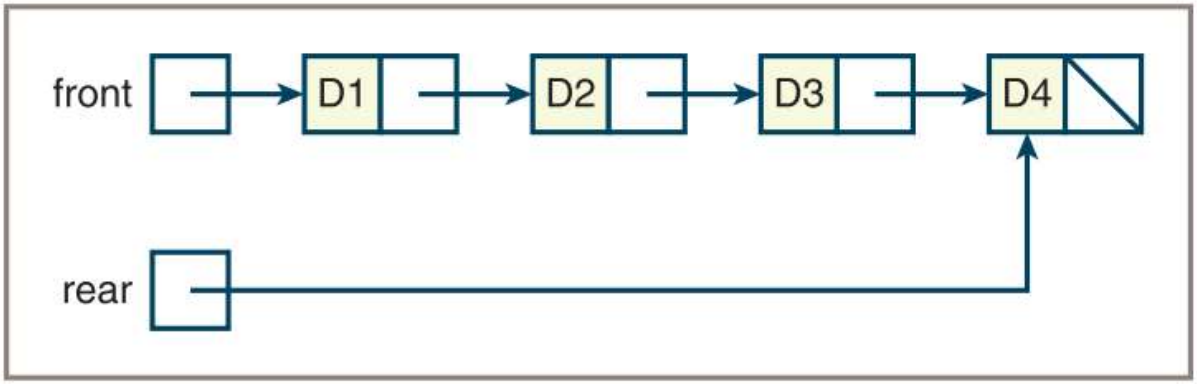

Figure 8-3 A linked queue with four items

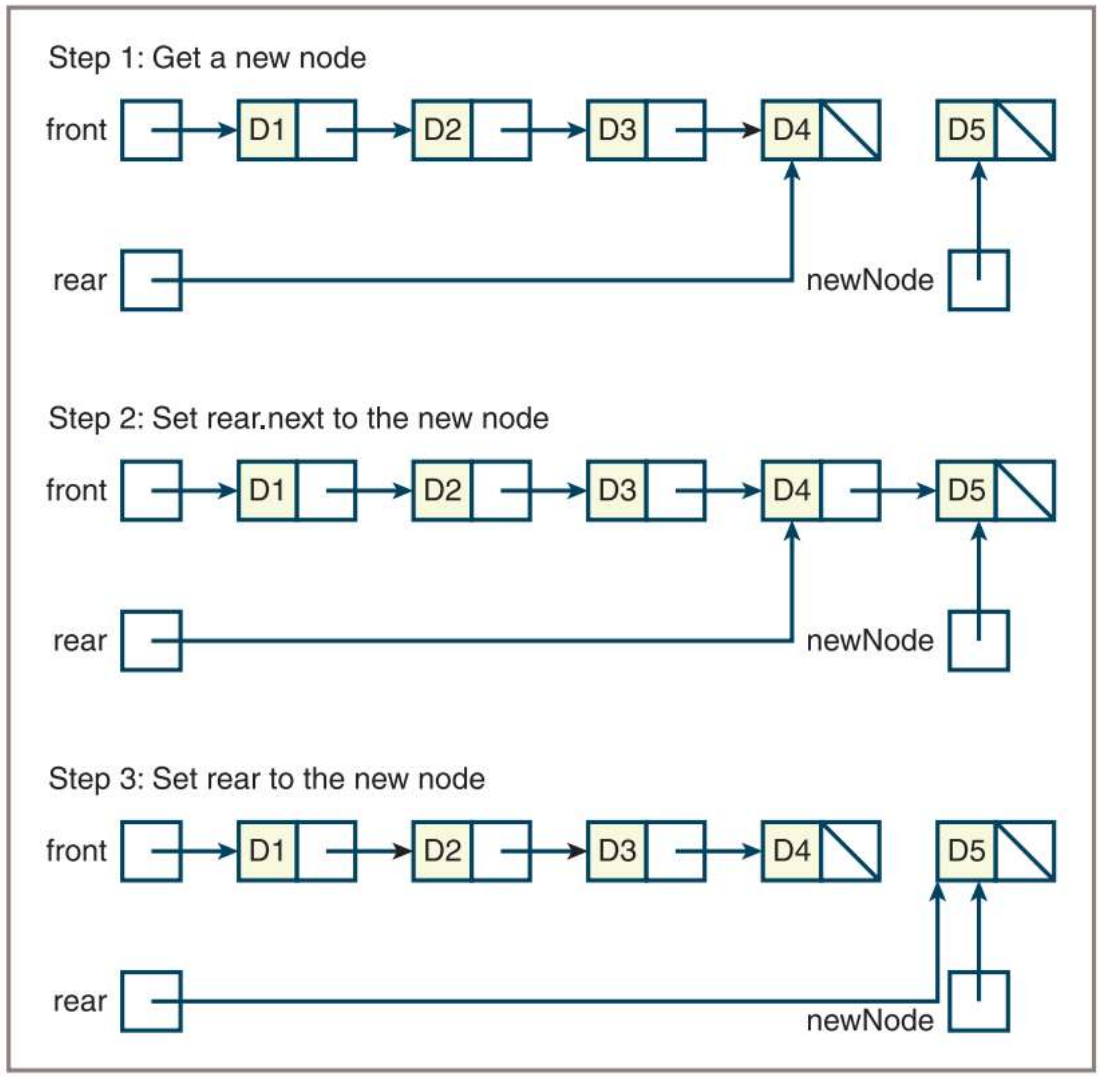

Figure 8-4 Adding an item to the rear of a linked queue

During an **add** operation, create a new node, set the next pointer of the last node to the new node, and set the variable **rear** to the new node, as shown in Figure 8-4.

```
def add(self, newItem):
    """Adds newItem to the rear of the queue."""
    newNode = Node(newItem, None)
    if self.isEmpty():
        self.front = newNode
    else:
        self.rear.next = newNode
    self.rear = newNode
    self.size += 1
```

```
def pop(self):
    """Removes and returns the item at front of the queue.
    Precondition: the queue is not empty."""
    # Check precondition here
    oldItem = self.front.data
    self.front = self.front.next
    if self.front is None:
        self.rear = None
    self.size -= 1
    return oldItem
```

An Array Implementation

- The first attempt at implementing a queue fixes the front of the queue at index position 0 and maintains an index variable, called rear, that points to the last item at position n - 1 where n is the number of items in the queue.

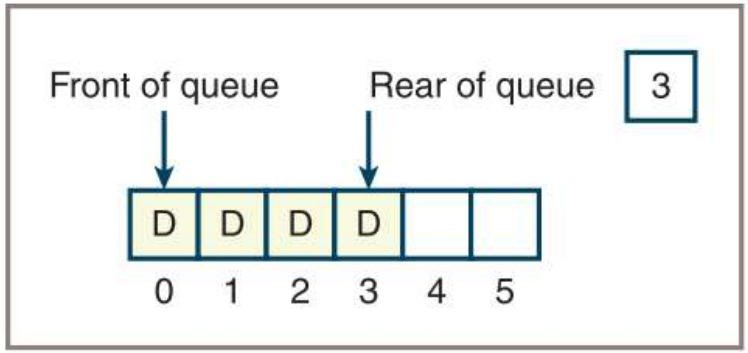

Figure 8-5 An array implementation of a queue with four items

An Array Implementation

- The first attempt at implementing a queue fixes the front of the queue at index position 0 and maintains an index variable, called rear, that points to the last item at position n - 1 where n is the number of items in the queue.

An Array Implementation

- Second Attempt: The front pointer starts at 0 and advances through the array as items are popped.

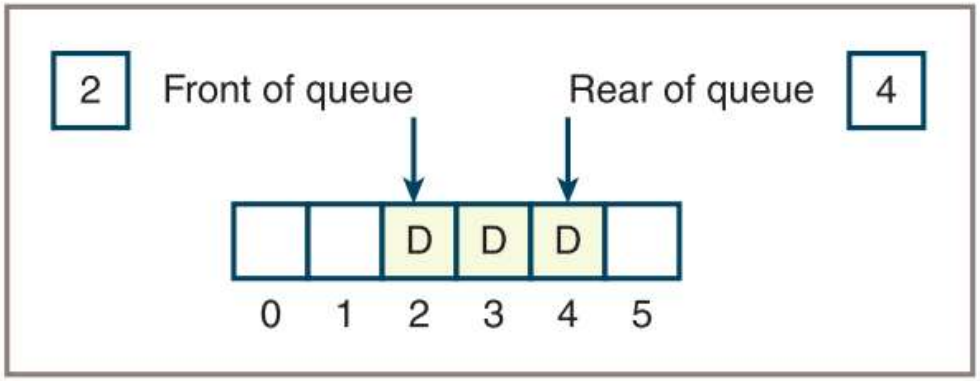

Figure 8-6 An array implementation of a queue with a front pointer

An Array Implementation

- Third Attempt: By using a circular array implementation, you can simultaneously achieve good running times for both add and pop.

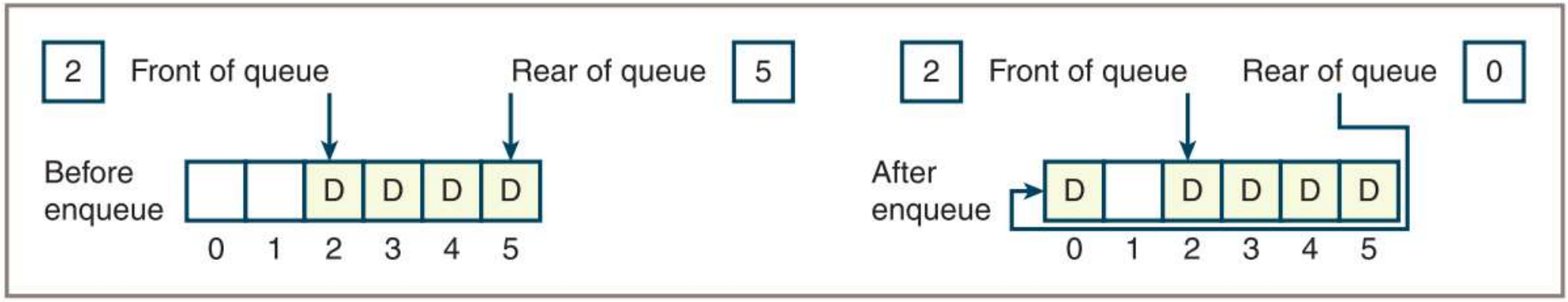

Figure 8-7 Wrapping data around a circular array implementation of a queue

Time and Space Analysis for the Two Implementations

- Linked implementation of queues
  - The running time of the **__str__**, **__add__**, and **__eq__** methods is O(*n*)
  - Running time of all the other methods is O(1)
- Circular array implementation of queues:
  - If the array is static, the maximum running time of all methods other than **__str__**, **__add__**, and **__eq__** is O(1)
  - If the array is dynamic, **add** and **pop** jump to O(*n*) anytime the array is resized but retain an average running time of O(1)

Exercises

1. Write a code segment that uses an **`if`** statement during an **`add`** to adjust the rear index of the circular array implementation of **`ArrayQueue`**. You may assume that the queue implementation uses the variables **`self.rear`** and **`self.items`** to refer to the rear index and array, respectively.

2. Write a code segment that uses the **`%`** operator during an **`add`** to adjust the rear index of the circular array implementation of **`ArrayQueue`** to avoid the use of an **`if`** statement. You can assume that the queue implementation uses the variables **`self.rear`** and **`self.items`** to refer to the rear index and array, respectively.

Priority Queues

- a specialized type of queue.
- When items are added to a priority queue, they are assigned an order of rank.
- When they are removed, items of higher priority are removed before those of lower priority.
- Queue and Priority Queue have the same interface or set of operations (see Table 8-1)

Operation	State of the Queue After the Operation	Value Returned	Comment
`q = <Priority queue type>()`			Initially, the queue is empty.
`q.add(3)`	3		The queue contains the single item 3.
`q.add(1)`	1 3		1 is at the front of the queue and 3 is at the rear of the queue because 1 has a higher priority.
`q.add(2)`	1 2 3		2 is added but has a higher priority than 3, so 2 moves ahead of 3.
`q.pop()`	2 3	1	Remove the front item from the queue and return it. 2 is now the front item.
`q.add(3)`	2 3 3		The new 3 is inserted after the existing 3, in FIFO order.
`q.add(5)`	2 3 3 5		5 has the lowest priority, so it goes to the rear.

Table 8-6 States in the Lifetime of a Priority Queue

```
class Comparable(object):
    """Wrapper class for items that are not comparable."""

    def __init__(self, data, priority = 1):
        self.data = data
        self.priority = priority

    def __str__(self):
    """Returns the string rep of the contained datum."""
        return str(self.data)

    def __eq__(self, other):
        """Returns True if the contained priorities are equal
        or False otherwise."""
        if self is other: return True
        if type(self) != type(other): return False
        return self.priority == other.priority
```

- E.g. we want to rank complex numbers by its magnitude/absolute value
- Priority = those values

```
def __lt__(self, other):
    """Returns True if self's priority < other's priority,
    or False otherwise."""
    return self.priority < other.priority

def __le__(self, other):
    """Returns True if self's priority <= other's priority,
    or False otherwise."""
    return self.priority <= other.priority

def getData(self):
    """Returns the contained datum."""
    return self.data

def getPriority(self):
    """Returns the contained priority."""
    return self.priority
```

```
queue.add(Comparable(a, 1))
queue.add(Comparable(b, 2))
queue.add(Comparable(c, 3))
while not queue.isEmpty():
    item = queue.pop().getItem()
    <do something with item>
```

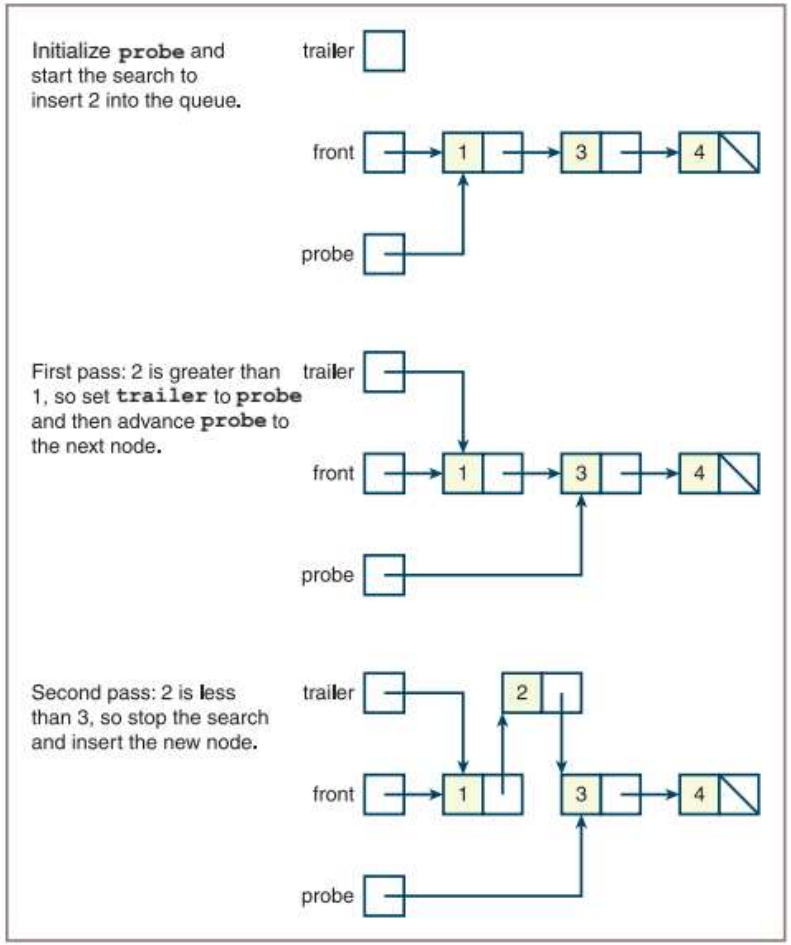

Figure 8-10 Inserting an item into a priority queue

```
"""
File: linkedpriorityqueue.py
"""

from node import Node
from linkedqueue import LinkedQueue

class LinkedPriorityQueue(LinkedQueue):
    """A link-based priority queue implementation."""

    def __init__(self, sourceCollection = None):
        """Sets the initial state of self, which includes the
        contents of sourceCollection, if it's present."""
        LinkedQueue.__init__(self, sourceCollection)

    def add(self, newItem):
        """Inserts newItem after items of greater or equal
        priority or ahead of items of lesser priority.
        A has greater priority than B if A < B."""
        if self.isEmpty() or newItem >= self.rear.data:
            # New item goes at rear
            LinkedQueue.add(self, newItem)
        else:
            # Search for a position where it's less
            probe = self.front
            while newItem >= probe.data:
                trailer = probe
                probe = probe.next
            newNode = Node(newItem, probe)
            if probe == self.front:
                # New item goes at front
                self.front = newNode
            else:
                # New item goes between two nodes
                trailer.next = newNode
            self.size += 1
```

Exercise

Suggest a strategy for an array-based implementation of a priority queue. Will its space/time complexity be any different from the linked implementation? What are the trade-offs?

Review Questions

1. Examples of queues are (choose all that apply):
   a. Customers waiting in a checkout line
   b. A deck of playing cards
   c. A file directory system
   d. A line of cars at a tollbooth
   e. Laundry in a hamper
2. The operations that modify a queue are called:
   a. Add and remove
   b. Add and pop
3. Queues are also known as:
   a. First-in first-out data structures
   b. Last-in first-out data structures
4. The front of a queue containing the items **a b c** is on the left. After two **pop** operations, the queue contains:
   a. **a**
   b. **c**
5. The front of a queue containing the items **a b c** is on the left. After the operation **add(d)**, the queue contains:
   a. **a b c d**
   b. **d a b c**

6. Memory for objects such as nodes in a linked structure is allocated on:
   a. The object heap
   b. The call stack
7. The running time of the three queue mutator operations is:
   a. Linear
   b. Constant
8. The linked implementation of a queue uses:
   a. Nodes with a link to the next node
   b. Nodes with links to the next and previous nodes
   c. Nodes with a link to the next node and an external pointer to the first node and an external pointer to the last node

9. In the circular array implementation of a queue:
   a. The front index chases the rear index around the array
   b. The front index is always less than or equal to the rear index
10. The items in a priority queue are ranked from:
   a. Smallest (highest priority) to largest (lowest priority)
   b. Largest (highest priority) to smallest (lowest priority)